

PROJECT REPORT

COMPUTER VISION PROJECT

NAME – RITIKA SINGH

REG NO.- 24BAI10190

SLOT – F11+F12

DATE OF SUBMISSION – 17 September 2026

INTRODUCTION AND THEORETICAL BACKGROUND

. Computer Vision Context: Algorithms convert unstructured matrix data to high-level spatial and contextual information.

. OCR: Optical Character Recognition engines binarize multi-channel image arrays to extract character shapes, and convert them into digital text.

. Gesture Key point Tracking: contemporary gesture tracking uses lightweight single-shot neural networks to localize spatial landmark points along human hand structures.

. Headless Systems Need: Implementing vision models in production server environments requires running from command line interface, to avoid desktop display dependencies.

THEORETICAL PROBLEM STATEMENT

. GUI Deprecation Problems: Common computer vision programs have a fundamental usage of GUI window output viewers, like `cv2.imshow()` which leads to fatal exceptions when run inside of an automated grading container pipeline.

- . Monolithic Disconnect: Typical computer vision applications disconnect text processing from structural feature discovery, requiring multi-framework bindings.
- . Solution Architecture: This project provides a monolithic python command line interface utility which unifies text extraction and hand gesture analysis operations together with structured logging and no GUI required.

SYSTEM REQUIREMENT ANALYSIS

FUNCTIONAL REQUIREMENTS

- . FR-1 (Ingestion Layer): Validate file path existence while decoding multi-channel imagery into 3D numpy arrays without segmentation faults.
- . FR-2 (Preprocessing Pipeline): Apply 5x5 Gaussian blur and adaptive Gaussian thresholding to correct for non-uniform illumination.
- . FR-3 (Text Extraction): Ingest binarized binary arrays into PyTesseract for clean text string extraction.
- . FR-4 (Gesture Detection): Localize 21 3D spatial hand joint landmarks using MediaPipe and calculate extended finger counts based on relative y-axis joint coordinates.
- . FR-5 (Task Dispatcher): Leverage argparse library to enable dynamic target routine selection (ocr, gesture, both) via command-line flags.

NON FUNCTIONAL REQUIREMENTS

- . NFR-1 (Processing Speed): The execution time of all steps for a single asset, including the image itself, takes less than 1.5 seconds on a CPU.
- . NFR-2 (Code Modularity): The application's source code is split into 6 different Python files that mostly perform one particular task each.

. NFR-3 (Fault Tolerance): The code has exception handlers that prevent the application from crashing if an error occurs during file reading, writing, or executing model inferences.

. NFR-4 (System Portability): The developed application can run on different command line interfaces, including Linux, MacOS, and Windows.

SYSTEM ARCHITECTURE AND COMPONENT DESIGN

Data Flow Pipeline

. Input Flag Parsing: CLI arguments define image asset paths and operational task selection.

. Array Validation: The Image Loader validates the target file path and decodes image bytes to a NumPy matrix array.

. Spatial Transformation: The Preprocessor standardizes image channels, suppresses high-frequency noise, and binarizes pixel values.

. Feature Extraction Execution: OCR Engine Branch: Binarized matrix text features are evaluated by PyTesseract.

. Gesture Engine Branch: Spatial keypoint landmark vectors are processed by MediaPipe.

. Output Dispatcher: Formatted text and numeric count output are printed to standard output stdout and logging diagnostics.

DESIGN DECISION AND TRADE-OFF ANALYSIS

. OpenCV Matrix Operations: Utilized the optimized C++ backend wrapped in NumPy operations.

. Adaptive Gaussian vs. Global Otsu Binarization: Adaptive thresholding looks at the distribution of pixel values in a local region, which makes it more effective at handling shadows in real-world documents than a global threshold.

. Media Pipe Landmark Estimator: Uses a lightweight single-shot detector model plus a key point regression model that can run on CPUs at real-time video processing speeds, without requiring GPU acceleration.

IMPLEMENTATION AND SOFTWARE STRUCTURE

. main.py: Controls the flow of execution and parses command line arguments. It directs the program's operation.src

. logger.py: Outputs logs with the same format.src/image_loader.py: Makes sure that files in the specified path exist and decodes the matrix byte data.src

. preprocessor.py: Converts the image's color to gray, applies Gaussian blur, and binarization based on an adaptive threshold.src

. ocr_engine.py: Uses wrapper classes around PyTesseract to extract text from images.src

. gesture_detector.py: Detects a hand gesture by comparing y coordinate values of all finger joints.tests

. test_pipeline.py: Tests the shape of a matrix array passed through a pipeline while using adaptive preprocessing, with pytest.

EXPERIMENTAL RESULTS AND DIAGNOSTICS

Processed sample images featuring document text and hand gestures under varied lighting.

ENGINEERING CHALLENGES AND SOLUTIONS

- . Uneven Image Illumination: Binarization of fixed documents was improved from using global thresholds to adaptive Gaussian algorithm to avoid shadow artifacts.
- . Headless GUI Exceptions: OpenCV imshow calls were removed to prevent crashes on the automated grading servers and redirect outputs to logging modules.

LEARNINGS AND FUTURE DIRECTION

Gained experience working with decoupled software design, image binarization, and landmark extraction in a command line interface.